

SYSTEM ARCHITECTURE DOCUMENT

PhishShield: A Machine Learning-Based Web Application for Phishing URL Detection

1. Introduction

1.1 Purpose

This System Architecture Document (SAD) describes the high-level structure, components, data flows, technology choices, and design decisions of the PhishShield web application. It serves as a blueprint for development, testing, and future maintenance of the system.

1.2 Scope

The document covers:

- Overall system architecture and layered design
- Major software components and their responsibilities
- Data flow for phishing URL detection
- Machine learning pipeline and feature extraction
- Technology stack and deployment considerations
- Security, performance, and scalability aspects

1.3 Project Overview

PhishShield is a lightweight, accessible web application that detects phishing URLs in real time using machine learning. Unlike traditional blacklist-based approaches that fail against newly created phishing sites, PhishShield analyzes lexical and structural features of a URL and classifies it as legitimate or phishing. Users simply enter a URL and receive an immediate prediction along with a security recommendation.

1.4 Key Objectives

- Develop a user-friendly web application for phishing URL detection
- Extract meaningful lexical and structural features from URLs
- Train and deploy a machine learning model on phishing and legitimate URL datasets
- Provide real-time prediction of whether a URL is safe or malicious
- Improve cybersecurity awareness among end users

2. System Overview

2.1 Architectural Style

PhishShield follows a layered (n-tier) architecture combined with a modular pipeline for the machine learning inference path. The primary style is a classic three-tier web application architecture augmented with a dedicated ML inference module:

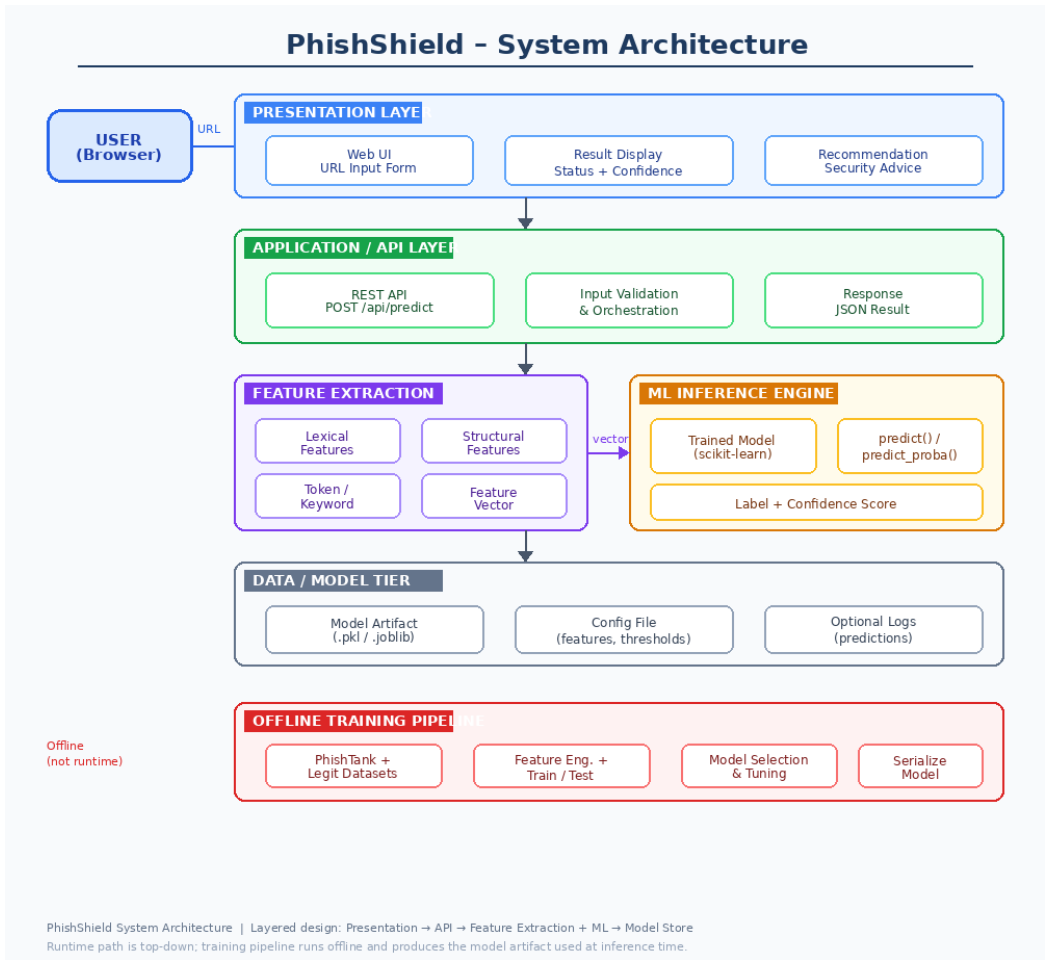
1. Presentation Tier – Web-based user interface
2. Application / Business Logic Tier – API, feature extraction, and prediction orchestration
3. Data / Model Tier – Trained ML model artifacts and optional logging store

2.2 High-Level Architecture

The system can be visualized as the following logical layers and components:

Layer	Components & Responsibilities
Presentation	Web UI (HTML/CSS/JS or template engine) – URL input form, result display, recommendation messages, responsive design
API Gateway / Backend	REST API endpoints, request validation, routing, response formatting, error handling
Feature Extraction	URL parsing, lexical feature computation (length, special characters, domain tokens, etc.), feature vector construction
ML Inference Engine	Loaded scikit-learn model, prediction, confidence score calculation
Model & Data Store	Serialized model file (.pkl / .joblib), optional prediction logs, configuration

Figure 1: PhishShield System Architecture Diagram



2.3 System Context

External actors and systems interacting with PhishShield:

- End User – Submits a URL via the web browser and receives a classification result
- Training Datasets (PhishTank, etc.) – Used offline during model training (not part of runtime)
- Browser – Renders the web interface; no browser extension is required in the current scope

3. Component Architecture

3.1 Presentation Layer

The presentation layer provides a clean, intuitive web interface accessible from any modern browser.

- URL Input Form – Single text field for pasting a complete URL (http/https)
- Submit / Check Button – Triggers the prediction request via AJAX or form POST
- Result Panel – Displays classification (Legitimate / Phishing), optional confidence score, and color-coded visual feedback
- Security Recommendation – Contextual advice (e.g., “Do not enter credentials on this site”)
- Responsive Layout – Works on desktop and mobile browsers without additional plugins

3.2 Application / Backend Layer

Implemented as a Python web service (Flask or FastAPI recommended for simplicity and ML ecosystem compatibility).

- API Endpoint (e.g., POST /api/predict) – Accepts URL string, returns JSON prediction
- Input Validation – Checks for empty input, basic URL syntax, length limits
- Orchestration Logic – Coordinates feature extraction → model inference → response assembly
- Error Handling – Graceful messages for invalid URLs or internal failures

3.3 Feature Extraction Module

This module transforms a raw URL string into a numerical feature vector suitable for the trained classifier. Typical URL-based features include:

Feature Category	Example Features
Lexical / Length	URL length, hostname length, path length, number of digits, number of special characters
Structural	Presence of IP address in hostname, number of subdomains, use of “@” or “//” in path, HTTPS presence
Token / Keyword	Suspicious keywords (login, secure, account, update, verify, bank, etc.), presence of hyphens
Domain-related	Top-level domain (TLD) type, domain entropy, ratio of consonants/vowels (optional advanced features)

Feature extraction is performed entirely on the server side so that the client never needs to implement complex logic. The resulting feature vector is passed directly to the inference engine.

3.4 Machine Learning Inference Engine

The core intelligence of the system resides in a supervised classification model trained offline on labeled phishing and legitimate URLs.

- Model Family – Classical ML algorithms via scikit-learn (e.g., Random Forest, Logistic Regression, Gradient Boosting, or SVM). Random Forest is a strong default due to robustness and feature-importance insights.
- Training Pipeline (Offline) – Dataset collection (PhishTank + legitimate sources) → feature extraction → train/test split → model selection & hyperparameter tuning → evaluation (accuracy, precision, recall, F1) → serialization
- Runtime Inference – Load serialized model at application start; for each request, transform features and call model.predict() / predict_proba()
- Output – Binary label (0 = Legitimate, 1 = Phishing) plus optional probability/confidence score

3.5 Data & Model Persistence

At runtime the system primarily needs the trained model artifact. Optional persistence includes:

- Model file stored on the server filesystem (joblib or pickle format)
- Lightweight logging of predictions (URL hash, timestamp, result) for monitoring and future retraining – can use SQLite or a simple log file
- Configuration (feature list, threshold, model path) stored in a YAML/JSON config file

4. Data Flow & Runtime Workflow

The end-to-end request flow for a single URL check is as follows:

1. User opens the PhishShield web page in a browser.
2. User pastes a URL into the input field and clicks “Check”.
3. Browser sends an HTTP POST request containing the URL to the backend API.
4. Backend validates the input (non-empty, reasonable length, basic URL format).
5. Feature Extraction Module parses the URL and computes the numerical feature vector.
6. ML Inference Engine feeds the feature vector into the loaded model and obtains the prediction (and optionally probability).
7. Backend packages the result into a JSON response (label, confidence, recommendation message).
8. Frontend receives the response and updates the UI with color-coded status and advice.
9. (Optional) Prediction metadata is logged for monitoring.

5. Technology Stack

Layer / Concern	Recommended Technology	Rationale
Backend Framework	Python + Flask or FastAPI	Native integration with scikit-learn; lightweight and fast for academic projects
Machine Learning	scikit-learn, pandas, numpy	Referenced in project proposal; mature, well-documented, suitable for classical models
Model Serialization	joblib / pickle	Standard for scikit-learn model persistence
Frontend	HTML5, CSS3, JavaScript (or Jinja2 templates)	Simple, no heavy SPA required; easy to keep lightweight
URL Parsing	urllib / tldextract / regex	Reliable extraction of domain, path, query components
Dataset Sources	PhishTank, open phishing/legitimate URL datasets	Aligned with project references
Deployment (Dev)	Local Python server / Flask development server	Rapid iteration during development
Deployment (Prod)	Gunicorn + Nginx, or cloud PaaS (Render, Railway, etc.)	Simple, low-cost hosting suitable for student projects

6. Deployment Architecture

A practical deployment topology for the academic version of PhishShield is:

1. Single-server deployment – Web server, application server, and model file reside on one host (or one container).
2. Static front-end assets (HTML/CSS/JS) served either by the Flask/FastAPI app or by a reverse proxy (Nginx).
3. Model loaded into memory at process start to avoid repeated disk I/O on every request.
4. Optional containerization with Docker for reproducible environments.

Because the system is lightweight and performs inference only (no heavy training at runtime), it can comfortably run on modest hardware or free-tier cloud instances.

7. Security Considerations

- Input Sanitization – Validate and sanitize all user-supplied URLs to prevent injection attacks.
- No Execution of Submitted URLs – The system never fetches or renders the target page; it only analyzes the URL string itself, eliminating the risk of the detector becoming a victim.
- HTTPS – Prefer serving the application over HTTPS in production so that user-submitted URLs are encrypted in transit.
- Rate Limiting – Simple rate limiting on the prediction endpoint to mitigate abuse.
- Model Integrity – Protect the serialized model file from unauthorized modification.

8. Non-Functional Requirements

8.1 Performance

Feature extraction and classical ML inference are computationally cheap. Target response time is under 500 ms for a single URL on typical student-project hardware, enabling a responsive user experience.

8.2 Scalability

The current design is sufficient for moderate concurrent usage. Horizontal scaling can later be achieved by running multiple application instances behind a load balancer, since the model is read-only and stateless per request.

8.3 Availability & Reliability

The system should return a clear error message rather than crash when an unexpected URL format is encountered. Health-check endpoints can be added for monitoring.

8.4 Maintainability

Clear separation of feature extraction, model loading, and API logic makes it straightforward to retrain the model and swap the artifact without rewriting the web layer. Configuration externalization further improves maintainability.

9. Alignment with Future Scope

The architecture intentionally leaves room for the enhancements listed in the project proposal:

1. Browser Extension – Can call the same prediction API; only a thin client needs to be added.
2. Mobile Application – Native or hybrid apps can consume the identical REST endpoint.
3. Content Analysis – An additional module can be inserted after feature extraction to fetch and analyze page content (with careful security controls).
4. Deep Learning Models – The inference interface can be swapped for a TensorFlow/PyTorch model while keeping the rest of the pipeline unchanged.

5. Continuous Model Updates – A background job or admin endpoint can reload a newly trained model artifact without downtime.
6. Multilingual Support – Primarily a UI concern; the core detection logic remains language-agnostic for URL features.

10. Summary

PhishShield is designed as a clean, layered web application that combines a simple user interface with a robust machine-learning inference pipeline. By focusing exclusively on URL-based features, the system remains lightweight, fast, and safe (no need to visit the potentially malicious site). The architecture supports the stated project objectives while remaining open to the future enhancements outlined in the proposal—browser extensions, mobile clients, deeper content analysis, and continuous model improvement.

This document provides the architectural foundation required for detailed design, implementation, testing, and subsequent academic evaluation of the PhishShield project.

Document Control

Version: 1.0 | Status: Initial Release | Date: August 2026
Prepared for: 3rd Year Project Idea Submission – PhishShield